

Programmable Data-Plane Service Chaining with P4 and Virtualized Network Functions

Miguel Carvalho, Solange Rito Lima, João F. Pereira, and Pedro Sousa

Department of Informatics, Centro Algoritmi/LASI
University of Minho
Braga, Portugal
pg57590@alunos.uminho.pt, solange@di.uminho.pt,
b12267@algoritmi.uminho.pt, pns@di.uminho.pt

Abstract. Traditional service chaining approaches rely on external control planes, static forwarding, or CPU-based middleboxes, which limit scalability and increase latency. Programmable data planes and the P4 language introduce a new model where traffic inspection and forwarding can be executed directly in the switch pipeline.

This work presents a fully functional service chaining platform based on P4 and Virtualized Network Functions (VNFs). The system integrates a stateful firewall, a load balancer, and a monitoring function. Traffic is routed through these functions without requiring centralized orchestration or software switching. The firewall uses hashing and Bloom filters to detect flow direction and maintain state. The load balancer distributes traffic among multiple backend servers using IP Virtual Server (IPVS). The monitoring function performs passive flow analysis and exposes performance metrics. The architecture is evaluated in a Containernet environment, using HTTP and TCP experiments. The results show that the programmable data-plane approach provides efficient traffic routing, supports multiple VNFs in a chain, and collects real-time performance information.

Keywords: SDN/NFV/VNF · Service Function Chaining · Programmable Data Planes · P4 · Firewall · Load Balancer · Performance Monitoring.

1 Introduction

Modern networks demand flexibility, programmability, and the ability to dynamically deploy network services. Traditional hardware and fixed-function network devices are not adequate for current traffic engineering requirements. Software-Defined Networking (SDN) and Network Function Virtualization (NFV) emerged as enablers of programmable networks [1, 2]. SDN allows centralized control and separation between the control and data planes as illustrated in Figure 1. NFV extends network services using software-based functions rather than dedicated hardware appliances. Programming Protocol-Independent Packet Processors (P4) language represents a step ahead, allowing developers to define packet parsing, matching, and forwarding directly in the switch pipeline [3].

Programmable NFV combines these concepts by executing part of the service logic inside the data plane and using virtualized functions only when necessary.

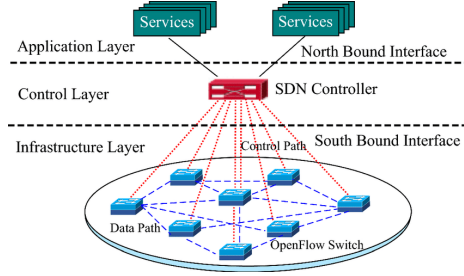


Fig. 1. Software-Defined Networks Architecture

In this paper, an NFV architecture based on P4 and virtualized functions is proposed and implemented, integrating a stateful firewall, a load balancer, and a monitoring service. Traffic is routed through these functions using P4, while the services run as containers in a Containernet environment. The solution performs packet processing, filtering, state tracking, and service chaining. The virtualized functions handle load balancing and traffic measurement. The proof-of-concept evaluates the developed prototype under real traffic conditions and demonstrates the correct operation of all components. Four load-balancing modes were considered and demonstrated stable and effective traffic distribution across the backend servers.

This paper is organized as follows: the related work is discussed in Section 2; the proposed service chaining architecture, including implementation details, is presented in Section 3; the system workflow is covered in Section 4; the evaluation is presented in Section 5; conclusions and future steps are drawn in Sections 6.

2 Related Work

Most NFV platforms still rely on CPU-based processing and external control systems, increasing overhead and limiting performance, particularly when traffic traverses multiple network functions [4, 5]. Recent works on NFV further analyze the challenges of deploying network functions efficiently in modern networks. In [2] and [6] it is highlighted that, although NFV improves flexibility by separating network functions from dedicated hardware, it introduces performance penalties due to software-based packet processing and frequent context switching. These works emphasize that service chaining across multiple Virtual Network Functions (VNFs) often leads to increased latency and higher resource consumption, particularly when traffic must cross several virtualized components. The works [7] and [8] provide a comprehensive overview of P4-based data plane programming and demonstrate how complex network functions can be implemented directly

within network switches. However, they also note that most existing implementations focus on isolated use cases, such as monitoring or load balancing, and lack an integrated approach that combines multiple services, virtualized functions, and centralized control within a unified architecture.

There is still a gap in solutions that integrate multiple functions into a single architecture. Most existing systems focus on only one service, such as load balancing or firewalling. The present work addresses this gap by integrating a programmable data plane and virtualized network functions into a single system.

3 System Architecture

This section describes the architecture of the implemented chaining system. The system is deployed in a virtual environment based on Containernet and BMv2, where: (i) Containernet is used to emulate the network topology with switches, routers, and hosts; (ii) BMv2 `simple_switch_grpc` is used as the P4 switch on all P4 nodes; (iii) all VNFs run as Docker containers instantiated inside Containernet. The VNF container image is based on Ubuntu 20.04.

3.1 Network Topology

As illustrated in Figure 2, the network is built as a core of P4 routers interconnected with an access switch and a service segment. In total, seven P4 switches are used:

- *switch1*: is a Layer-2 (L2) learning switch for the access network;
- *router1*: is the edge router and entry point to the service chain (P4 routing, tunnels, and VMon);
- *router2*, *router3*, *router5*, *router6*: are core routers running the tunnel P4 program (label-based forwarding);
- *router4*: is an edge router and firewall.

The routers *router2*, *router3*, *router5*, and *router6* parse MSLP labels (see details in Section 3.2), match on the top label, forward to the next hop, and pop labels as the packet progresses through the service tunnel. In the service segment, the Virtual Load Balancer (VLB) container (VLoadBalancer) behaves as an integrated L3/L4 node. It has multiple interfaces, maintains routes for the Virtual IP (VIP) and backend subnets, and runs IP Virtual Server (IPVS) in Network Address Translation (NAT) mode to distribute traffic across backend servers. The load balancer performs routing and flow-level classification at the host level, complementing the P4 data-plane logic. The VMon container (Virtual Monitor) is attached to *router1* and *router4* and operates purely as a passive monitoring VNF. It receives cloned packets, reconstructs flows, and exports metrics.

Overall, most of the packet classification, path selection, tunneling, firewalling, and service routing is implemented inside the P4 switches (*switch1*, *router1* - *router6*). The VLB VNF adds L3/L4 routing and load-balancing logic for the service segment, while VMon provides out-of-band monitoring.

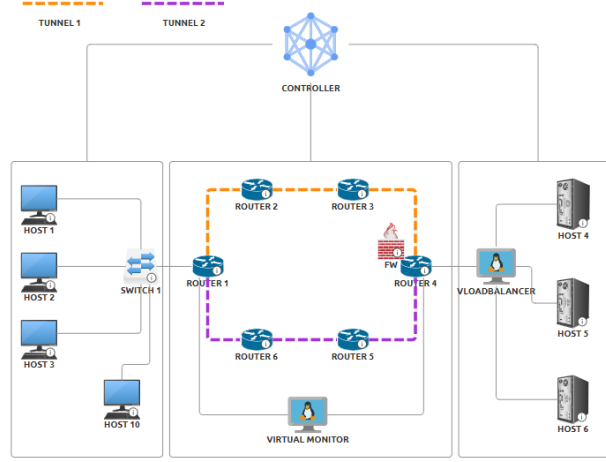


Fig. 2. Network Topology for testing the Data-Plane Service Chaining

3.2 MSLP Tunneling

The service chain is implemented using a tunnel format named *Multipurpose Sequence Label Protocol* (MSLP), inspired by Multiprotocol Label Switching (MPLS) but designed specifically for P4. Unlike traditional encapsulation approaches (e.g., VXLAN or MPLS), MSLP is entirely implemented inside the P4 data plane. The tunnel is represented as a label stack that identifies the next service in the chain. Each label corresponds to a service function, and the forwarding decision is performed only on the top label, while the next-hop Medium Access Control (MAC) address determines the physical path to reach that service.

The MSLP header is inserted directly in the ingress pipeline. When a service chain is required, a new label stack is written into the packet:

```

action addMSLP(bit<64> labels) {
    hdr.mslp.setValid();
    hdr.labels[0] = { labels[63:48], 0x00 };
    hdr.labels[1] = { labels[47:32], 0x00 };
    hdr.labels[2] = { labels[31:16], 0x00 };
    hdr.labels[3] = { labels[15:0], 0x01 };
}

```

Listing 1.1. MSLP Label Insertion

The VNF never sees or processes labels. Routing is executed entirely inside the switches. Each router in the core implements a simple label-based forwarding table. Switching between services is achieved using an exact-match table:

```

table labelLookup {
    key = {

```

```

        hdr.labels[0].label : exact;
    }
    actions = {
        forwardTunnel;
        removeMSLP;
        drop;
    }
}

```

Listing 1.2. Table LabelLookup

This design enables multiple service stages without changing the original packet and without re-parsing L3 headers at every hop. Overhead is minimal, since only a small stack of labels is inserted.

When the packet reaches the last service, the tunnel is terminated, and the original packet is restored by removing the label header. This architecture provides full data-plane programmability while keeping VNFs simple and stateless. The core network only interprets labels and forwards the packet.

3.3 P4 Forwarding Pipeline

All switches follow the same P4 processing model: (i) Parser; (ii) Ingress pipeline; (iii) Egress pipeline; (iv) Deparser, further described below. Even though the model is the same, each switch uses the pipeline differently, according to its role in the chain (routing, tunneling, firewalling, tunnel load balancing).

Parser The parser extracts the main protocol headers and identifies whether the packet is part of the service chain. The supported protocols include Ethernet, IPv4, ICMP, TCP, UDP, and MSLP labels for tunneling. For encapsulated packets, the parser iterates over the labels until the Bottom-of-Stack bit is found.

Ingress Pipeline The ingress pipeline contains the main logic for forwarding and service routing. The pipeline performs routing, firewall checks, policy enforcement, or tunnel selection. The core routing decision is implemented with a standard LPM table:

```

table ipv4Lpm {
    key = {
        hdr.ipv4.dstAddr : lpm;
    }
    actions = {
        forward;
        sendTunnel;
        drop;
    }
}

```

Listing 1.3. Table IPv4LastPrefixMatch

If the packet is not routed normally, it is sent to the service chain.

Egress Pipeline The egress pipeline applies final packet transformations based on the forwarding decision. Typical operations include MAC rewrite, tunnel label pop, recirculation for multi-stage processing, or cloning for monitoring. For example, a clone is generated when sending a packet to the VMon system:

```
clone_preserving_field_list (CloneType.I2E, 99, 0);
```

Listing 1.4. Clone Action of VMon

Deparser The deparser emits the packet with headers reordered and updated according to the pipeline result. This includes removing or inserting labels and updating Ethernet fields.

3.4 Firewall Switch (FW)

The firewall is implemented as a P4 router and represents a mandatory stage in the service chain. It is connected to the VLB VNF and forms the network boundary between the core and the service segment. Unlike traditional NFV architectures, the firewall logic is integrated directly into the data plane.

The switch performs three main functions in the ingress pipeline: packet classification, direction detection, and flow validation. Direction is identified by matching the ingress and egress ports:

```
table checkDirection {
    key = {
        meta.ingress_port          : exact;
        standard_metadata.egress_spec : exact;
    }
}
```

Listing 1.5. Table CheckDirection

This enables the firewall to detect whether a packet belongs to the forward or reverse direction of a flow, which is fundamental for enforcing asymmetric rules. Instead of maintaining a connection table or flow records, the firewall uses a compressed state representation based on two Bloom filters of 4096 entries each:

```
register<bit<1>>(4096) bloom1;
register<bit<1>>(4096) bloom2;
```

Listing 1.6. Bloom Filter

If a reverse flow does not find its corresponding bits, the packet is dropped. This allows the firewall to preserve awareness without maintaining per-flow state or allocating memory proportional to the number of connections. Traffic is inspected and filtered based on protocol semantics: UDP flows are allowed only

on explicitly configured ports; TCP flows are allowed if they belong to the VIP service or to a load-balanced backend flow; ICMP is blocked when returning to the client side.

All rules are executed within the P4 pipeline. The firewall does not rely on the VLB or VMon containers and does not forward packets to a CPU. The firewall marks the transition from the core forwarding plane into the service chain. It implements access control, state validation, and protocol filtering before the VLB VNF is reached.

3.5 Tunnel Load Balancer

The switch *router1* acts as an edge load balancer for the service chain. Unlike classical NFV designs that implement load balancing inside a VNF, the balancing decision at *router1* is computed entirely in the P4 pipeline. The switch uses a deterministic 5-tuple hash over the packet fields to select the service chain path:

```
hash(meta.tunnel, HashAlgorithm.crc32,
    {hdr.ipv4.srcAddr,
     hdr.ipv4.dstAddr,
     hdr.tcp.srcPort,
     hdr.tcp.dstPort},
    2);
```

Listing 1.7. Tunnel Algorithm

The goal of this design is not to select the backend server directly, but to select the service path through the core. The flow is therefore mapped to a particular MSLP label stack, which determines the path towards the core. This architecture has several advantages: (i) load-balancing logic is performed in the data plane; (ii) flows are deterministic and symmetric; (iii) forwarding latency remains minimal; (iv) no controller involvement is required. After the hash is computed, routing to the next stage is performed by inserting the appropriate MSLP label stack.

3.6 Virtual Load Balancer (VLB)

The VLB is deployed as a standalone container that receives traffic directly from *router4*. It performs backend selection using the Linux IPVS framework. IPVS is used as the load-balancing engine running inside the VLB container. It operates at Layer-4 and performs connection-oriented dispatching of TCP and UDP flows. IPVS maintains a VIP associated with multiple backend servers and selects one of them for every new connection according to a scheduling algorithm. It supports NAT mode, in which packets destined to the VIP are rewritten and forwarded to the selected backend server. After the backend responds, IPVS performs the reverse translation, preserving connection transparency.

The VLB supports multiple scheduling algorithms, including: *Round Robin* - Distributes flows sequentially across all backends; *Least Connections* - Selects

the backend node with the fewest active connections; *Weighted Round Robin* - Supports configured weights; *Source-Hashing* - Maintains connection stickiness for specific clients. These algorithms operate at the VNF level, while flow routing into the VLB is performed entirely by *r4*. The VLB runs in NAT mode, meaning that the clients are unaware of backend IPs. The VLB provides effective load balancing without removing the programmability of the data plane.

3.7 Monitoring VNF (VMon)

The VMon container is a passive monitoring VNF. It doesn't forward traffic, and it is not on the critical path. Instead, traffic is mirrored using the P4 cloning mechanism at *router1* and *router4*. The VMon then performs flow extraction using 5-tuple, and computes per-flow byte/packet statistics, Round-Trip Time (RTT) for TCP flows, jitter, and detects retransmissions. The VMon also runs an HTTP API that exports live metrics and logs, making all measurements externally observable.

4 Data-Plane Service Chaining Workflow

The Data-Plane service chaining prototype presented in this work operates as a pipeline that combines a data-plane service routing with lightweight exemplificative VNFs (see Figure 2). When a packet enters the network through *switch1*, the switch performs basic L2 learning and replication. The packet is forwarded to *router1*, where the P4 parser identifies the protocol stack and checks for MSLP encapsulation. If the packet is native IPv4 traffic, *router1* performs a routing or service chain decision using the *ipv4Lpm* table. Traffic that requires processing is routed into the tunnel by attaching a MSLP label stack. The label identifies the next service stage. All forwarding decisions in the core are taken only using the top label. The next-hop MAC is used only to reach the next switch and does not represent the service itself. This ensures symmetric load balancing across different service paths without keeping state in the controller.

Packets then traverse the firewall switch (*router4*), which performs stateful inspection and protocol filtering. It validates direction and flow consistency using compressed state stored in Bloom Filters and applies policy enforcement for TCP, UDP, and ICMP traffic. When the firewall logic approves the flow, *router4* forwards the packet to the appropriate VNF segment. At this point, the VLB and VMon containers operate in parallel, without the latter impacting traffic forwarding. Traffic is forwarded to the VLB module for backend selection, where IPVS in NAT mode distributes flows across the backend servers. The three servers (*host4*, *host5*, and *host6*) are directly connected to the VLB through three independent subnets: 10.0.4.0/30, 10.0.5.0/30 and 10.0.6.0/30.

Each server on such networks runs an HTTP service on port 81 and an *iperf* service on port 5001. The virtual IP exposed to clients is 10.0.2.1, and *router4* acts as the gateway 10.0.2.254. In parallel, the monitoring process receives traffic clones from both *router1* and *router4* using P4 clone sessions. This VNF

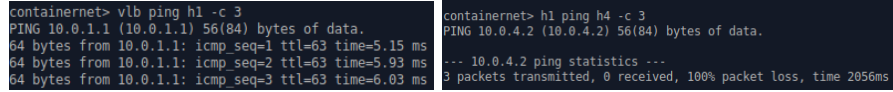
monitoring process passively processes packets without participating in forwarding decisions, extracting measurements such as RTT, jitter, retransmissions, flow direction, and per-flow statistics. The monitoring logic does not require packet modification or buffering actions.

5 Experimental Evaluation and Results

This section evaluates the system under real traffic conditions and demonstrates the correct operation of all components in this illustrative, yet advanced, service chain architecture. The focus is to provide simple but concrete evidences of all the service modules achieving the planned objectives (i.e., firewall, load balancer, monitoring, and tunneling).

5.1 Connectivity Control

To validate the firewall policy, we probed ICMP connectivity in both directions. When the VLB container pings *host1* (10.0.1.1), all echo requests are successful, confirming that traffic from the server side towards the client network is permitted. In contrast, a direct ping from *host1* to backend *host4* (10.0.4.2) results in 100% packet loss. This behaviour matches the P4 firewall programming logic on *r4*, where the `checkDirection` table marks traffic entering from the client side and going to the backend subnets as inbound (`direction = 1`) and the ICMP policy explicitly drops echo requests (`icmp.type = 8`) in this direction. Figure 3 confirm this behavior that was programmed in the firewall.



```

containernet> vlb ping h1 -c 3
PING 10.0.1.1 (10.0.1.1) 56(84) bytes of data:
64 bytes from 10.0.1.1: icmp_seq=1 ttl=63 time=5.15 ms
64 bytes from 10.0.1.1: icmp_seq=2 ttl=63 time=5.93 ms
64 bytes from 10.0.1.1: icmp_seq=3 ttl=63 time=6.03 ms

containernet> h1 ping h4 -c 3
PING 10.0.4.2 (10.0.4.2) 56(84) bytes of data:
--- 10.0.4.2 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2056ms

```

Fig. 3. (left) Ping From VLB to *host1*; (right) Failed ping from *host1* to *host4*

5.2 Evaluation of Backend Load-Balancing Behavior

The load balancer was tested using four different algorithms: Round Robin (RR), Weighted Round Robin (WRR), Source Hashing (SH) and Least Connections (LC). The results presented in Table 1 show that the system distributes traffic correctly and consistently, according to the selected strategy and established setup.

Round Robin (RR): RR produces a nearly perfect distribution of flows among the three backend servers. As observed in Table 1, each server node processes roughly the same number of flows and packets. The *throughput* and *pps* values are almost identical across the server nodes, demonstrating fairness.

Weighted Round Robin (WRR): WRR shows different balancing pattern, with different proportions in accordance with the configured parameters. The backend server with the highest weight receives more flows, as expected. This proves that the weighting mechanism is correctly interpreted by IPVS inside the VLB container. As expected, the variation in *Mbps* and *pps* is larger than in RR, which is consistent with the dynamic scheduling nature of the WRR mechanism.

Source Hashing (SH): The SH method provides a very similar distribution of flows across the three servers. All the three backend servers receive almost the same number of connections and produce nearly identical *throughput* and *pps* values. It is important to note that this hash-based method do not requires runtime state, which is why this mechanism is interesting to be used in high demanding scenarios. These results shows that, despite the VNF does not maintain any session tables or per-flow metadata, the balancing is fully deterministic and performed solely based on the flow 5-tuple.

Least Connections (LC): The LC method produces slightly different flow counts, but the overall distribution of the traffic remains balanced across all the servers. This method results are aligned with the ones obtained by the SH and RR in order to obtain a uniform distribution of requests among the backend servers.

All four load-balancing modes demonstrated stable and effective traffic distribution across the backend servers. The throughput remains consistent across tests, and each server receives a share of packets and bytes in accordance with the configured mechanisms.

The results presented in Table 1 also integrate distinct metrics. The *RTT* values extracted from TCP acknowledgments show that the end-to-end latency across the entire service chain is stable. *Minimum RTT* values remain low, and the *average RTT* stays similar across backend servers, indicating that the forwarding paths are symmetric. The *jitter* values, computed as variations in consecutive RTT samples, remain similar across all backends and algorithms. Although jitter is relatively high due to virtualization of the used emulation framework and parallel traffic arrivals, the important remark to be noted is that it remains balanced and does not diverge between backend servers.

5.3 VMon - Virtual Monitoring Service

In the developed service chain architecture filtering rules were validated using UDP and ICMP traffic. The firewall correctly blocks flows that target the backend servers in the wrong direction and only allows flows that follow the intended request-response pattern. At the same time, the monitoring VNF (VMon) detects and reports suspicious traffic both at *router1* and *router4*. To confirm the correct inspection made by this monitoring service along the entire service chain, it was programmed to consider UDP traffic as suspicious. In the captured logs (see Figure 4 (left)) the VMon service module alerts the network administrator when deteting unexpected UDP packets arriving at the backend network.

Table 1. Load Balancing Performance Metrics [Source Hashing (SH), Round Robin (RR), W. Round Robin (WRR), Least Connections (LC)]

Load Bal.	Backend	Flows	Pkts	Bytes	Dur (s)	pps	Mbps	Retr.	RTT avg (ms)	RTT min (ms)	Jitt. (ms)
SH	10.0.4.2	8476	53710	14405316	162.607	330.31	0.709	22277	2264.259	0.001	6147.095
	10.0.5.2	8712	54709	15063734	163.341	334.94	0.738	22652	2152.021	0.001	6263.832
	10.0.6.2	8832	55884	15792028	162.912	343.03	0.775	22806	2206.179	0.001	6269.729
RR	10.0.4.2	1608	6165	584818	110.850	55.62	0.042	4191	3626.446	0.002	4793.357
	10.0.5.2	1608	6086	539864	110.869	54.89	0.039	4173	3645.824	0.002	4846.075
	10.0.6.2	1608	6062	510112	110.762	54.73	0.037	4216	2814.400	0.003	4845.571
WRR	10.0.4.2	1578	8514	2055612	99.033	85.97	0.166	4168	2301.269	0.001	4962.773
	10.0.5.2	4742	25741	6306902	99.183	259.53	0.509	12426	2188.729	0.001	4927.538
	10.0.6.2	7920	42521	10272770	99.500	427.35	0.826	20622	2196.623	0.001	4779.536
LC	10.0.4.2	2622	10453	982626	215.569	48.49	0.036	7157	4567.138	0.001	6050.203
	10.0.5.2	2596	10358	997376	214.812	48.22	0.037	7042	4982.800	0.002	6648.321
	10.0.6.2	2606	10395	1011670	214.807	48.39	0.038	7047	4773.296	0.002	6087.664

5.4 Tunnel Load Balancing

The tunnel switch maintains independent traffic counters for each service chain direction. The values illustrated in Figure 4 (right) confirm correct bidirectional traversal.

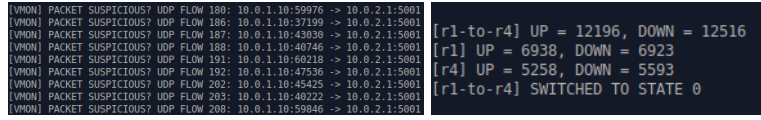


Fig. 4. (left) VMon detecting suspicious UDP traffic; (right) Tunnel Load Balancing

6 Conclusions and Future Research Directions

This work presented a P4-based service chaining platform integrating virtualized network functions, including a stateful firewall, a load balancer, and a monitoring module. The architecture enables traffic steering directly in the programmable data plane without centralized orchestration. Experimental evaluation in a *Containernet* environment demonstrated efficient routing, support for multiple chained VNFs, and real-time performance monitoring.

While the proposed architecture successfully and effectively implements service chaining, in-band tunneling, stateful firewalling, and a monitoring VNF, several extensions would benefit the system. A relevant future step is completing the feedback loop between the data plane, VMon, and the controller. Presently, VMon detects unwanted traffic and extracts complete flow metadata, but this information is not consumed automatically by the network. A natural improvement is enabling the controller to react to monitoring events and enforce new rules dynamically. For example, when VMon identifies suspicious traffic, the controller could install new firewall entries in *router4* or update tunnel policies in

router1 in real time. Another research direction is the comparison between two load-balancing approaches: the current container-based VLB and a P4 implemented load balancer. A P4 implementation could reduce latency and eliminate context switching overhead, while the current Linux-based VLB offers flexibility and simple deployment. Future work may evaluate both models using the same traffic conditions and quantify differences in throughput, latency, and fairness. Finally, the architecture can be extended with VNF scaling. VMon analytics can be used to trigger dynamic instantiation of new VNF containers when throughput exceeds a threshold or when an imbalance is detected between backend servers.

Acknowledgement

This work has been supported by FCT - Fundação para a Ciência e a Tecnologia, within the R&D Unit Project Scope UID/00319/2025 - Centro ALGORITMI (ALGORITMI/UM), URL: <https://doi.org/10.54499/UID/00319/2025>.

References

1. D. Kreutz, F. M. V. Ramos, P. E. Veríssimo, C. E. Rothenberg, S. Azodolmolky, and S. Uhlig, "Software-Defined Networking: A Comprehensive Survey," *Proc. IEEE*, vol. 103, no. 1, pp. 14–76, Jan. 2015, doi: 10.1109/JPROC.2014.2371999.
2. B. Han, V. Gopalakrishnan, L. Ji, and S. Lee, "Network Function Virtualization: Challenges and Opportunities for Innovations," *IEEE Communications Magazine*, vol. 53, no. 2, pp. 90–97, Feb. 2015, doi: 10.1109/MCOM.2015.7045396.
3. P. Bosshart, D. Daly, G. Gibb, M. Izzard, N. McKeown, J. Rexford, C. Schlesinger, D. Talayco, A. Vahdat, G. Varghese, and D. Walker, "P4: Programming Protocol-Independent Packet Processors," *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 3, pp. 87–95, Jul. 2014, doi:10.1145/2656877.2656890.
4. A. Laghrissi and T. Taleb, "A Survey on the Placement of Virtual Resources and Virtual Network Functions," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 2, pp. 1409–1434, 2019, doi: 10.1109/COMST.2018.2884835.
5. P. Quinn and T. Nadeau, "Problem Statement for Service Function Chaining," *IETF RFC 7498*, Apr. 2015. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7498.html>
6. R. Mijumbi, J. Serrat, J.-L. Gorricho, N. Bouten, F. De Turck, and R. Boutaba, "Network Function Virtualization: State-of-the-Art and Research Challenges," *IEEE Communications Surveys & Tutorials*, vol. 18, no. 1, pp. 236–262, First Quarter 2016, doi: 10.1109/COMST.2015.2477041.
7. F. Hauser, M. Häberle, D. Merling, S. Lindner, V. Gurevich, F. Zeiger, R. Frank, and M. Menth, "A Survey on Data Plane Programming with P4: Fundamentals, Advances, and Applied Research," *Journal of Network and Computer Applications*, vol. 212, Art. no. 103561, Mar. 2023, doi: 10.1016/j.jnca.2022.103561.
8. E. F. Kfoury, J. Crichigno, and E. Bou-Harb, "An Exhaustive Survey on P4 Programmable Data Plane Switches: Taxonomy, Applications, Challenges, and Future Trends," *IEEE Access*, vol. 9, pp. 87094–87155, 2021, doi: 10.1109/ACCESS.2021.3086704.